

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

|                  |                                                                                                                       |               |                                                   |
|------------------|-----------------------------------------------------------------------------------------------------------------------|---------------|---------------------------------------------------|
| Course Code:     | CSA06                                                                                                                 | Course Title: | Design and Analysis of Algorithms                 |
| CO Assessed:     | CO2 – Manipulation (BL1, BL2, BL3)                                                                                    | Assessment:   | Assessment Tool 3 – Debugging & Optimisation Task |
| Weightage:       | 10% of CIA                                                                                                            | Total Marks:  | 100                                               |
| CO2 Statement:   | <i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i> |               |                                                   |
| Student Name:    | P.sai charan tej                                                                                                      | Reg. No.:     | 192465048                                         |
| Section / Batch: | CSE-2024                                                                                                              | Date:         | 28-07-26                                          |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

| Question Type                 | Focus Area                                                                                      | Questions |
|-------------------------------|-------------------------------------------------------------------------------------------------|-----------|
| Debugging & Optimisation Task | Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques | Q1        |

## SECTION A – DEBUGGING & OPTIMISATION TASK

### Q43. Debugging Linear Search Code (Incorrect Loop Starting Index)

The following Linear Search implementation begins checking from the wrong index and may skip a valid match at the start of the array. Carefully analyze the loop initialization and explain why the first element is ignored. Apply debugging to ensure that every element from the beginning of the array is checked correctly. Optimize the implementation for simplicity and readability while preserving correct functionality. Analyze the time complexity of the corrected version and rewrite the final code with proper comments.

```
def linear_search(arr, key):
    for i in range(1, len(arr)): # ERROR
        if arr[i] == key:
            return i
    return -1
```

### Code of Conduct

I G Lokesh Yadav -192411061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Lokesh Yadav

## RUBRICS FOR CALCULATION

| Criteria               | Wt. | Level 4 – Exemplary                                                                  | Level 3 – Proficient                                   | Level 2 – Developing                          | Level 1 – Beginning                           |
|------------------------|-----|--------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Problem Identification | 20% | Accurately identifies all errors and root causes with clear understanding            | Identifies most errors with minor gaps                 | Identifies some errors but misses key issues  | Unable to identify errors correctly           |
| Debugging              | 25% | Applies systematic debugging techniques effectively to resolve issues                | Uses appropriate debugging methods with minor errors   | Limited debugging approach; requires guidance | Unable to debug or resolve issues             |
| Optimization           | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity        | No improvement; inefficient approach retained |
| Analysis               | 20% | Demonstrates strong logical reasoning and clear justification of solutions           | Shows reasonable analysis with some justification      | Limited analysis; weak reasoning              | No analytical reasoning demonstrated          |
| Code Quality           | 15% | Produces clean, well-structured code with proper comments and documentation          | Code is mostly clear with minor documentation gaps     | Code lacks structure and proper documentation | Poorly written code with no documentation     |

### Marks Evaluation:

| Criteria                  | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Problem Identification    | 20% |                     |                      |                      |                     |
| Debugging                 | 25% |                     |                      |                      |                     |
| Optimization              | 20% |                     |                      |                      |                     |
| Analysis                  | 20% |                     |                      |                      |                     |
| Code Quality              | 15% |                     |                      |                      |                     |
| <b>Total Marks (100):</b> |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name: _____       | Name: _____        | Name: _____        |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## Q. Debugging Pair Sum Code (Redundant Comparisons in Nested Loops)

### Given Code

```
for i in range(len(arr)):
    for j in range(len(arr)):
        if arr[i] + arr[j] == target:
            return (i, j)
```

### 1. Inefficiencies in the Loop Structure

- The inner loop always starts from index 0.
- It compares an element with itself ( $\text{arr}[i] + \text{arr}[i]$ ).
- It checks duplicate pairs such as  $(i, j)$  and  $(j, i)$ .
- These repeated comparisons increase unnecessary computations and reduce efficiency.

### 2. Why Duplicate Comparisons Occur

Example:

$\text{arr} = [2, 4, 6]$

$\text{target} = 8$

The algorithm checks:

$(2,2)$

$(2,4)$

$(2,6)$

$(4,2) \leftarrow$  Duplicate of  $(2,4)$

$(4,4)$

$(4,6)$

$(6,2) \leftarrow$  Duplicate of  $(2,6)$

$(6,4) \leftarrow$  Duplicate of  $(4,6)$

$(6,6)$

Here,

- $(2,4)$  and  $(4,2)$  represent the same pair.
- $(2,6)$  and  $(6,2)$  are duplicate comparisons.
- $(4,6)$  and  $(6,4)$  are duplicates.
- $(2,2)$ ,  $(4,4)$ , and  $(6,6)$  compare an element with itself.

Hence, many unnecessary comparisons are performed.

### 3. Debugged (Improved) Code

```
def pair_sum(arr, target):
```

```
    n = len(arr)
```

```
    # Check each unique pair only once
```

```
    for i in range(n):
```

```
for j in range(i + 1, n):  
    if arr[i] + arr[j] == target:  
        return (i, j)
```

```
return None
```

#### 4. Why This Version is Better

- The inner loop starts from  $i + 1$ .
- Avoids comparing an element with itself.
- Eliminates duplicate pair comparisons.
- Each unique pair is checked exactly once.
- Produces the same correct result with fewer comparisons.
- Improves readability while preserving correctness.

#### 5. Complexity Analysis

Original Algorithm

Outer Loop:

$n$

Inner Loop:

$n$

Total Comparisons:

$n \times n = n^2$

Time Complexity:

$O(n^2)$

#### Improved Algorithm

Outer Loop:

$n$

Inner Loop:

$n-1, n-2, n-3, \dots, 1$

Total Comparisons:

$n(n-1) / 2$

Time Complexity:

$O(n^2)$

Although the Big-O complexity remains  $O(n^2)$ , the improved version reduces the number of comparisons from  $n^2$  to  $n(n-1)/2$ , making it significantly more efficient.

#### 6. Sample Execution

$arr = [2, 7, 11, 15]$

$target = 9$

Output

$(0, 1)$

Because

$2 + 7 = 9$

## Output:-

```
1
2
3 def linear_search(arr, key):
4
5     for i in range(len(arr)):
6         if arr[i] == key:
7             return i
8     return -1
9
10
11 arr = [10, 20, 30, 40, 50]
12
13 key = int(input("Enter the element to search: "))
14
15 result = linear_search(arr, key)
16
17 if result != -1:
18     print("Element found at index:", result)
19 else:
20     print("Element not found")
```

input

```
Enter the element to search: 20
Element found at index: 1
```

## 7. Final Answer

The original algorithm performs unnecessary comparisons by checking self-pairs and duplicate pairs, resulting in extra computations. By modifying the inner loop to start from  $i + 1$ , each unique pair is examined only once. This eliminates redundant comparisons while maintaining correctness. The optimized algorithm still has a time complexity of  $O(n^2)$ , but it reduces the total number of comparisons from  $n^2$  to  $n(n-1)/2$ , making it more efficient and easier to understand.